

РОССИЙСКИЙ УНИВЕРСИТЕТ ДРУЖБЫ НАРОДОВ

Факультет физико-математических и естественных наук

Кафедра прикладной информатики и теории вероятностей

ОТЧЕТ

ПО ЛАБОРАТОРНОЙ РАБОТЕ № 01

Геометрические фракталы

Студент: Стелина Петрити

Группа: НПИбд-02-21

Геометрические фракталы

Задание: написать компьютерную программу для построения геометрического фрактала «Дерево Пифагора». Программа должна соответствовать следующим требованиям:

- За построение фрактала должна отвечать функция `void pifagor(int n, int x0, int y0, int a, double fi, double alpha)`, в которой
 - `n` – глубина рекурсии построения фрактала,
 - `x0, y0` – растровые координаты точки привязки «домика»,
 - `a` – длина основания «домика»,
 - `fi` – угол наклона основания «домика»,
 - `alpha` – угол наклона крыши «домика»;
- Функция `pifagor` должна рекурсивно вызывать себя два раза с фактическими параметрами, соответствующими задаче построения двух «домиков» на скатах крыши. В программе должна быть установлена максимальная глубина рекурсии;
- В функции `pifagor` должны вычисляться растровые координаты других, помимо точки привязки, вершин «домика»;
- При построении фрактала должна использоваться функция `line` графической библиотеки.

Код

```
#include <iostream>
#include <SFML/Graphics.hpp>
#include <cmath>
// Определение числа pi
#define pi 3.1415926536
using namespace sf;
// Начальные параметры для построения дерева
int a = 100;           // Длина основной ветки
int n = 12;            // Уровень рекурсии
float alpha = pi / 3;  // Угол между ветками

//Эта функция принимает объект окна рендеринга, координаты двух точек (x1,
//y1, x2, y2) и цвет. Он использует SFML для рисования линии между
//указанными //точками заданным цветом.

void drawLine(RenderWindow& window, int x1, int y1, int x2, int y2,
sf::Color color) {
    Vertex line[] = {
        Vertex(sf::Vector2f(x1, y1), color),
        Vertex(sf::Vector2f(x2, y2), color)
    };
    window.draw(line, 2, sf::Lines);
}
```

*//Эта рекурсивная функция рисует дерево Пифагора. Он принимает окно,
//глубину рекурсии n, начальную точку (x0, y0), длину ветви a, угол
//fi, угол альфа и параметр цвета.*

```
void pifagor(RenderWindow& window, int n, int x0, int y0, int a, float
fi, float alpha, int linecolor) {
    float zx = a * sin(fi);float zy = a * cos(fi);
    int x1 = x0 + zx;  int x2 = x0 + zx - zy;
    int y1 = y0 - zy;  int y2 = y0 - zy - zx;
    int x3 = x0 - zy;  int y3 = y0 - zx;
    int x4 = x0 - zy + a * cos(alpha) * sin(fi - alpha); int y4 = y0 -
zx - a * cos(alpha) * cos(fi - alpha);
```

*//Функция вычисляет новые точки на основе алгоритма дерева Пифагора
//и рисует линии между ними с помощью функции DrawLine. Он также
//рекурсивно вызывает сам себя с обновленными параметрами для рисования
//меньших ветвей.*

```
    drawLine(window, x0, y0, x1, y1, Color(linecolor * 10, 0, 0));
    drawLine(window, x1, y1, x2, y2, Color(linecolor * 10, 0, 0));
    drawLine(window, x2, y2, x3, y3, Color(linecolor * 10, 0, 0));
    drawLine(window, x3, y3, x0, y0, Color(linecolor * 10, 0, 0));
```

```
    drawLine(window, x3, y3, x2, y2, Color(linecolor * 10, 0, 0));
    drawLine(window, x2, y2, x4, y4, Color(linecolor * 10, 0, 0));
    drawLine(window, x4, y4, x3, y3, Color(linecolor * 10, 0, 0));
```

```
    if (n > 1) {
//первая ветвь
        pifagor(window, n - 1, x4, y4, a * sin(alpha), fi - alpha + pi
/ 2, alpha, linecolor + 1);
//вторая ветвь
        pifagor(window, n - 1, x3, y3, a * cos(alpha), fi - alpha,
alpha, linecolor + 1);
    }
}
```

```
int main() {
    sf::RenderWindow window(VideoMode(1000, 1000), "Pifagor's tree");
    while (window.isOpen()) {
// Обработываем очередь событий в цикле
        sf::Event event;
        while (window.pollEvent(event)) {
            if (event.type == sf::Event::Closed)
                window.close();//закрываем его
        }
// Установка цвета фона
    }
```

```

        window.clear(sf::Color::White);
        pifagor(window, n, 600, 750, a, pi / 2, alpha * 1 / 2, 1);
// Отрисовка окна
window.display();
    }
    return 0;
}

```

Результат работы

